claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-a67f7d609fa770fef.jsonl`
- SHA-256: `99795aeec3926201f2c84b40779765a75a096274e111b3b230909af0dc30d624`
- Source modified: `2026-07-06T17:30:35+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf_133acdee-dab`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T17:26:48.060Z)

```
You are a CONVENTIONS & TEST-QUALITY reviewer.
REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
DIFF UNDER REVIEW (read this first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff

GOAL OF THE CHANGE (wf294 incident): the Cloud Run rally-worker reported a run as
status="success" with 0 segments even when the WASB shuttle-trajectory detector TIMED OUT
(1800s) and produced zero trajectory. A detector timeout/abort was indistinguishable from a
legitimately 0-rally video. The fix must make "detector failed/timed out" surface as a
failed status (NOT "success") in report.json AND the completion marker, while a genuine
0-rally match stays a success.

WHAT THE FIX DID (4 files, all in the diff):
1. backend/pipeline/segmenters/trajectory_hybrid.py ? process_video() now returns a
   backend.results.Failure (instead of a bare []) on TWO detector-failure paths: the env
   healthcheck-not-ready path, and the "run_predict produced no trajectory" abort path.
   A trajectory-produced-but-zero-windows case still returns bare [] (a legit 0-rally).
   Return annotation widened to Union[List[Dict[str,Any]], Failure]; kept the
   # type: ignore[override].
2. backend/worker/__main__.py ? added _serialize_and_push_outputs() helper (writes+pushes
   intervals.csv + report.json). _fail(rc) now writes+pushes a status="failed" report.json /
   intervals.csv (0 segments), best-effort, BEFORE the M6 done-marker. The success path uses
   the same helper. The segmenter-Failure branch already routed to _fail(3).
3. tests/test_tracknet_hybrid.py ? the two abort tests now assert isinstance(res, Failure);
   added a test that zero-windows stays a bare [] (not a Failure).
4. tests/test_worker.py ? new test that a segmenter Failure -> rc 3 + report.json
   status="failed" + done-marker returncode 3/status="failed"; strengthened the 0-segment
   test to assert status="success".

KEY CONSUMER CODE TO CROSS-CHECK (read the real files, do not assume):
- backend/worker/__main__.py: cmd_infer (the full control flow: validation -> segmenter ->
  isinstance(result, Failure) -> _fail(3); the finally: emit_indexer_report; the success tail),
  _write_done_marker, _write_report_json, _write_intervals_csv.
- backend/orchestration.py: cached_process_result (~line 288, requires report status=="success"
  to hydrate), probe_process_cache (~line 255), _read_bucket_json, _resume_one_remote_job usage.
- backend/compute/cloud_run.py: CloudRunCompute.run_infer (reads the done-marker returncode +
  outputs/<id>/report.json).
- backend/api/job_worker.py: _resume_one_remote_job (waits for report.json to EXIST, then reads
  status; marks the job failed if status != success), _run_claimed_job.
- backend/results.py: Success/Failure/Result/unwrap_or_failure contract.
- backend/main.py (isinstance Failure branch ~1557), backend/pipeline/segmenters/gemini.py &
  motion.py (the existing Failure precedent), yolo_hybrid.py (a PARALLEL hybrid segmenter).
```

CONSTRAINTS THE FIX MUST RESPECT: ruff clean, mypy clean (trajectory_hybrid is NOT quarantined; worker.__main__ IS quarantined via mypy.ini ignore_errors), full suite green, coverage floor 70%.
The full suite is already green (1736 passed) and ruff/mypy clean ? so do NOT report "tests might fail" speculatively; report only concrete defects you can point at in the code.

Only report REAL defects: a wrong/edge behavior, a silent-failure path still leaking as success, a control-plane interaction regression, an idempotency/concurrency bug, a broken test assertion, or a genuine convention violation. Cite exact file:line. If a concern is speculative or you cannot ground it in the actual code, DO NOT report it.

YOUR LENS, part A (conventions): Does the change follow repo conventions?
- status token: the code uses "failed". The task said "failed/partial". Is "failed" consistent with every consumer (they check =="success", and DB/marker use "failed")? Any place expecting "partial"?
- Failure(is_recoverable=True) on both detector paths ? is that semantically right (retryable) and consistent with gemini/motion usage?
- The widened return annotation Union[List,Failure] + kept # type:ignore[override]: correct? Do the docstring/comments accurately describe the new behavior? Any stale comment left (e.g. the old "override-ignore: every live segmenter returns a bare list")?
- Twin/base parity: process_video is the shared base for wasb/tracknet/fusion ? confirm the fix is in the base (applies to all three) and no subclass overrides it.
YOUR LENS, part B (tests): Do the new/updated tests actually LOCK the behavior and cover the right things?
- test_worker: does it assert BOTH report.json status=="failed" AND the marker returncode/status?
  Does it cover the NO --done-uri failure case (report still failed, no marker)? Does it distinguish rc 3 (detector) from rc 2 (validation)?
- Is there an END-TO-END worker test that a real trajectory-hybrid Failure (not just a fake _FailSeg) flows through cmd_infer? Is the healthcheck-failure path covered via the worker, not only the segmenter unit test?
- test_tracknet: are the Failure assertions specific (message/is_recoverable) not just truthiness?
  Is the zero-windows legit-empty test asserting run_predict WAS called (a real run)?
- Any assertion that is too loose, or a gap that would let a regression pass silently?
Report concrete convention violations or test gaps only, with file:line.

2. assistant (2026-07-06T17:26:54.223Z)

I'll review this change. Let me start by reading the diff and the key consumer files.

3. user (2026-07-06T17:26:55.834Z)

```
1       diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2       index 503f6d8..f96b495 100644
3       --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4       +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5       @@ -22,7 +22,7 @@ import concurrent.futures
6        import logging
7        import os
8        import time
9       -from typing import Any, Dict, List, Optional, Tuple
10      +from typing import Any, Dict, List, Optional, Tuple, Union
11
12       import cv2
13
14      @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15       from backend.pipeline.detectors.base import DetectorRunner
16       from backend.pipeline.segmenters.base import BasePlaySegmenter
17       from backend.providers import provider_registry
18      +
19      +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20      +from backend.results import Failure
21       from backend.sports import sport_registry
```

```
 from backend.utils.paths import output_base
 from backend.utils.video import extract_video_chunk, get_video_duration
@@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         video_path: str,
         player_pool: Optional[List[str]] = None,
         max_frames: Optional[int] = None,
-    ) -> List[Dict[str, Any]]:
-        # override-ignore: the ABC declares the Result contract (Success|Failure)
-        # but every live segmenter still returns a bare list ? the documented
-        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
+    ) -> "Union[List[Dict[str, Any]], Failure]":
+        # Partial adoption of the Result contract (the ABC declares Success|Failure).
+        # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
+        # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
+        # genuine 0-rally video (which still returns a bare ``[]``). Every caller
+        # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
+        # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
         label = self.display_label
         # --- Sport profile + AI provider wiring (identical across segmenters) ---
         sport_name = self.config.sport
@@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         ok, msg = runner.healthcheck()
         if not ok:
             logger.error("%s detector environment not ready: %s", label, msg)
-            return []
+            # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
+            # failed/retryable instead of writing an empty-but-"successful" result.
+            return Failure(
+                message=f"{label} detector environment not ready: {msg}",
+                is_recoverable=True,
+            )
         logger.info("%s detector env OK: %s", label, msg)

         # --- Phase 2: trajectory -> candidate rally windows ---
@@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
         if not csv_path:
             logger.error("%s produced no trajectory; aborting.", label)
-            return []
+            # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
+            # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
+            # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
+            # rather than a silent 0-segment "success" (the wf294 incident).
+            return Failure(
+                message=(
+                    f"{label} detector produced no trajectory (timed out or aborted) ?
"
+                    "detector failure, not a 0-rally video."
+                ),
+                is_recoverable=True,
+            )

         points = runner.parse_trajectory_csv(csv_path)
         logger.info(
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 461d7ca..88c50f6 100644
--- a/backend/worker/__main__.py
```

```
      +++ b/backend/worker/__main__.py
      @@ -114,6 +114,42 @@ def _write_report_json(
               )


      +def _serialize_and_push_outputs(
      +    scratch: str,
      +    out_uri: str,
      +    video_id: str,
      +    segments: List[dict],
      +    status: str,
      +    storage_cfg: dict,
      +    video_uri: str = "",
      +) -> List[str]:
      +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and push both to
      +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
      +
      +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector timeout/abort leaves a
      +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
      +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any non-``success``
      +    report as "no usable result" ? retry, and the restart-recovery poll waits for report.json to
      +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
      +    out_local = os.path.join(scratch, "outputs", video_id)
      +    os.makedirs(out_local, exist_ok=True)
      +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
      +    _write_report_json(
      +        video_id,
      +        segments,
      +        status,
      +        os.path.join(out_local, "report.json"),
      +        video_uri=video_uri,
      +    )
      +    out_prefix = out_uri.rstrip("/")
      +    pushed: List[str] = []
      +    for fn in sorted(os.listdir(out_local)):
      +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
      +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
      +        pushed.append(uri)
      +    return pushed
      +
      +
       def _run_startup_checks(config: Any) -> bool:
           """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA
           best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
      @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
           )

           def _fail(rc: int) -> int:
      -        # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments) so a remote
      -        # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until its poll
      -        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
      +        # A worker-side FAILURE (validation, unknown segmenter, or a detector timeout/abort the
      +        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success". Write + push
      +        # a status="failed" report.json/intervals.csv (0 segments) so the control
```

```
plane's bucket
132     +        # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133     +        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134     +        # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
135     +        # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136     +        try:
137     +            _serialize_and_push_outputs(
138     +                scratch,
139     +                args.out_uri,
140     +                video_id,
141     +                [],
142     +                "failed",
143     +                storage_cfg,
144     +                str(getattr(args, "video_uri", "") or ""),
145     +            )
146     +        except Exception as e:  # never mask the real failure on an artifact-push
hiccup
147     +            logger.warning("[worker] could not push failure artifacts: %s", e)
148     +        # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149              if getattr(args, "done_uri", None):
150                  _write_done_marker(
151                      args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152      @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153              )
154
155          # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156      -    out_local = os.path.join(scratch, "outputs", video_id)
157      -    os.makedirs(out_local, exist_ok=True)
158      -    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159      -    _write_report_json(
160      +    pushed = _serialize_and_push_outputs(
161      +        scratch,
162      +        args.out_uri,
163              video_id,
164              segments,
165              status,
166      -        os.path.join(out_local, "report.json"),
167      -        video_uri=str(getattr(args, "video_uri", "") or ""),
168      +        storage_cfg,
169      +        str(getattr(args, "video_uri", "") or ""),
170          )
171
172      -    out_prefix = args.out_uri.rstrip("/")
173      -    pushed = []
174      -    for fn in sorted(os.listdir(out_local)):
175      -        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176      -        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177      -        pushed.append(uri)
178      -
179          print(
180              f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181          )
182      diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183      index 64909e7..c5ae442 100644
184      --- a/tests/test_tracknet_hybrid.py
185      +++ b/tests/test_tracknet_hybrid.py
186      @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187       from unittest.mock import MagicMock, patch
188
```

```
189      from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190     +from backend.results import Failure
191
192
193      def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(
195
196          seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197          res = seg.process_video("v1", "clip.mp4")
198      -    assert res == []
199      +    # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200      +    # instead of writing an empty "success".
201      +    assert isinstance(res, Failure)
202      +    assert "environment not ready" in res.message
203          # Should bail before ever running inference.
204          mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207          mock_provider_registry.get.return_value = MagicMock()
208
209          seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210      -    assert seg.process_video("v1", "clip.mp4") == []
211      +    res = seg.process_video("v1", "clip.mp4")
212      +    # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213      +    # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214      +    assert isinstance(res, Failure)
215      +    assert "no trajectory" in res.message and res.is_recoverable
216      +
217      +
218      +@_patched
219      +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220      +    mock_runner_cls,
221      +    mock_probe,
222      +    mock_dur,
223      +    mock_extract,
224      +    mock_provider_registry,
225      +    mock_env,
226      +):
227      +    """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228      +    parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229      +    (0-rally "success"), NOT a Failure."""
230      +    mock_env.return_value = "dummy_key"
231      +    mock_dur.return_value = 30.0
232      +    mock_extract.return_value = True
233      +    mock_runner_cls.return_value = _make_runner(windows=[])  # trajectory ok, zero
windows
234      +    mock_provider_registry.get.return_value = MagicMock()
235      +
236      +    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237      +    res = seg.process_video("v1", "clip.mp4")
238      +    assert res == [] and not isinstance(res, Failure)
239      +    # The detector DID run (a genuine empty, not an abort).
240      +    mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243      @_patched
244     diff --git a/tests/test_worker.py b/tests/test_worker.py
245     index 96edbee..63cc18e 100644
246     --- a/tests/test_worker.py
247     +++ b/tests/test_worker.py
```

```
    @@ -10,6 +10,7 @@ import json
     import pytest

     from backend.config import load_config
    +from backend.results import Failure
     from backend.worker import __main__ as wmain

     # ---------------------------------------------------------------- pure helpers
    @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):


     class _EmptySeg:
    -    """A segmenter that finds nothing ? the cold-start failure mode."""
    +    """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

         def __init__(self, db, config):
             pass
    @@ -200,6 +201,20 @@ class _EmptySeg:
             return []


    +class _FailSeg:
    +    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
    +    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
    +
    +    def __init__(self, db, config):
    +        pass
    +
    +    def process_video(self, video_id, path, max_frames=None):
    +        return Failure(
    +            message="WASB detector produced no trajectory (timed out or aborted)",
    +            is_recoverable=True,
    +        )
    +
    +
     def test_setup_logging_levels():
         import logging

    @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
         assert "0 segments" in msgs and "vidZ" in msgs
         assert "detector_impl=native" in msgs  # surfaces the resolved detector
         assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
    -    assert (
    -        json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
    -            "segment_count"
    +    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
    +    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
    +    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
    +    assert report["segment_count"] == 0 and report["status"] == "success"
    +
    +
    +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
    +    tmp_path, monkeypatch
    +):
    +    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
    +    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
    +    status="failed" so the control plane / alpha-user surface can show an error /
retry."""
    +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
```

```
306     +    monkeypatch.setattr(
307     +        wmain.validator_registry,
308     +        "run_all_with_context",
309     +        lambda path, cfg: ([_OKResult()], {}),
310     +    )
311     +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
312     +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313     +    video = tmp_path / "in.mp4"
314     +    video.write_bytes(b"FAKE")
315     +    out = tmp_path / "bucket"
316     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317     +
318     +    rc = wmain.main(
319     +        [
320     +            "infer",
321     +            "--video-uri",
322     +            str(video),
323     +            "--out-uri",
324     +            str(out),
325     +            "--scratch",
326     +            str(tmp_path / "s"),
327     +            "--segmenter",
328     +            "wasb_hybrid",
329     +            "--done-uri",
330     +            str(done),
331              ]
332     -        == 0
333          )
334     +    # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335     +    assert rc == 3
336     +    # report.json is written as an explicit FAILURE, not a false empty "success".
337     +    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338     +    assert report["status"] == "failed" and report["segment_count"] == 0
339     +    assert report["segments"] == []
340     +    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341     +    marker = json.loads(done.read_text())
342     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
343     +    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346      def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347
```

4. assistant (2026-07-06T17:27:00.961Z)

Let me read the key consumer files to cross-check the fix.

5. user (2026-07-06T17:27:02.766Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
```

```
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
62
63      def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64          """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66          - ``Failure``            -> ``(None, failure)``
67          - ``Success(value=v)``   -> ``(v, None)``
68          - a bare value (legacy code that predates this contract and still returns a raw
list)
69            -> ``(value, None)``
70
71          This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72          that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73          on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
```

```
74          (Success | Failure | legacy bare value)."""
75          if isinstance(result, Failure):
76              return None, result
77          value = result.value if hasattr(result, "value") else result
78          return value, None
79
```

6. user (2026-07-06T17:27:02.823Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      from typing import Any, List
28
29      from backend import storage
30      from backend.compute import ComputeJobSpec, get_compute_backend
31      from backend.config import (
32          StartupCheckError,
33          enforce_startup_checks,
34          load_config,
35          probe_cuda_available,
36      )
37      from backend.infrastructure.database import Database
38      from backend.pipeline.segmenters.base import segmenter_registry
39      from backend.pipeline.validators.base import registry as validator_registry
40      from backend.reporting import emit_indexer_report
41      from backend.results import Failure
42      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("backend.worker")
46
47
48      def _coerce(v: str) -> Any:
49          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50          low = v.lower()
51          if low in ("true", "false"):
```

```
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )
```

```
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
```

```
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-dispatch.

        A remote job that never writes a completion marker (a failed/hung container) makes the bucket
        poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote dispatch
        timeout/failure, distinct from the local 2/3) rather than letting a raw traceback escape."""
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri,
            out_uri=args.out_uri,
            segmenter=args.segmenter,
            config_overrides=tuple(args.set or ()),
        )
        try:
            result = compute.run_infer(spec)
        except PolicyTimeout as e:
            logger.warning(
                "[worker] remote compute did not complete (no marker within budget): %s", e
            )
            return 4
        logger.info(
            "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
            result.video_id,
            result.returncode,
            result.outputs_uri,
        )
        return result.returncode


    def _write_done_marker(
        done_uri: str,
        video_id: str,
        returncode: int,
        segment_count: int,
        status: str,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success return."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "segment_count": segment_count,
                "status": status,
            }
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
```

```
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(
            local_video, config
        )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
```

```
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
                )
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(
                video_id, local_video, max_frames=args.max_frames
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
```

```
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0
```

```
394
395
396     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
397     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
398     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
399
400     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
401     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
402     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
403     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
404
405
406     def _label_clip_name(fname: str) -> str:
407         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
408
409         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
410         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
411
412         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
413         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
414         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
415         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
416
417         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
418         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
419         """
420         name = fname.replace(".rallies.csv", "").replace(".csv", "")
421         return _LABEL_DATE_PREFIX.sub("", name)
422
423
424     def _probe_video_fps_width(path: str):
425         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
426
427         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
428         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
429         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
430         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
431         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
432         rather than guesses.
433         """
434         import json
435         import subprocess
436
437         try:
438             out = subprocess.check_output(
439                 [
440                     ffprobe_bin(),
441                     "-v",
```

```
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
```

```
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(
            u
            for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
            if u.lower().endswith(".csv")
        )
        if len(label_uris) < 2:
            print(
                f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                f"under {corpus}/labels/"
            )
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
```

```
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
            local_label = storage.fetch(
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
```

```
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
```

```
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.set_defaults(func=cmd_infer)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
```

```
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
            "--segmenter",
            default="wasb_hybrid",
            help="trajectory hybrid whose detector is used for --trajectory-source
detector",
        )
        pt.add_argument(
            "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
        )
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument(
            "--fps",
            type=float,
            default=30.0,
            help="synth trajectory fps (detector probes the video)",
        )
        pt.add_argument(
            "--frame-width",
            type=float,
            default=1280.0,
            help="synth trajectory frame width (detector probes the video)",
        )
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
        entry), not at import, so importing the module never reconfigures a caller's
logging."""
        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
        # configured logging at import), which would leave the level at WARNING and re-
suppress the
        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
        logging.basicConfig(
            level=logging.DEBUG if verbose else logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%H:%M:%S",
            force=True,
        )


    def main(argv=None) -> int:
        args = build_parser().parse_args(argv)
        _setup_logging(getattr(args, "verbose", False))
        return args.func(args)


    if __name__ == "__main__":
        sys.exit(main())

```

7. assistant (2026-07-06T17:27:05.267Z)

Now let me read the orchestration, cloud_run, job_worker, and the segmenter files to cross-check consumer behavior.

8. user (2026-07-06T17:27:06.789Z)

```
230         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
232         ref = storage.parse_uri(uri)
233         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
234             local = storage.fetch(
235                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
236             )
237             with open(local, encoding="utf-8") as f:
238                 loaded = json.load(f)
239         return loaded if isinstance(loaded, dict) else None
240
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
243     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
244     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
245     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
246     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
247     _REHYDRATE_LOCKS_GUARD = threading.Lock()
248
249
250     def _rehydrate_lock(video_id: str) -> threading.Lock:
251         with _REHYDRATE_LOCKS_GUARD:
252             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
253
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
257
258         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
259         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
260         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
261         errors report False (? the normal dispatch path with its guards)."""
262         try:
263             existing = db.get_video(video_id)
264             if (
265                 existing
266                 and existing.get("status") == "processed"
267                 and db.query_play_segments(video_id, {})
268             ):
269                 return True
270             sc = _storage_settings(cfg)
271             if not sc or (sc.get("backend") or "local") == "local":
272                 return False
273             output_root = (sc.get("output_root") or "").rstrip("/")
274             if not output_root:
275                 return False
276             report_uri = f"{output_root}/outputs/{video_id}/report.json"
277             if not storage.exists(report_uri, config=sc):
278                 return False
279             report = _read_bucket_json(report_uri, sc)
280             return bool(report and report.get("status") == "success")
281         except Exception:  # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
```

```
path
282             logger.warning(
283                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284             )
285             return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
295         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
297         partial earlier ingest is replaced, never double-counted.
298
299         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
301         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
302         job before invoking (writes happen post-claim)."""
303         try:
304             with _rehydrate_lock(video_id):
305                 existing = db.get_video(video_id)
306                 if existing and existing.get("status") == "processed":
307                     segs = db.query_play_segments(video_id, {})
308                     if segs:
309                         return _cached_process_payload(
310                             video_id,
311                             existing.get("validation_results", []),
312                             segs,
313                             requested_compute,
314                         )
315                 sc = _storage_settings(cfg)
316                 if not sc or (sc.get("backend") or "local") == "local":
317                     return None
318                 output_root = (sc.get("output_root") or "").rstrip("/")
319                 if not output_root:
320                     return None
321                 outputs_uri = f"{output_root}/outputs/{video_id}"
322                 report_uri = f"{outputs_uri}/report.json"
323                 if not storage.exists(report_uri, config=sc):
324                     return None
325                 report = _read_bucket_json(report_uri, sc)
326                 if not report or report.get("status") != "success":
327                     return None
328                 if not video_path:
329                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330                     # carries the worker's --video-uri (P3); older reports fall back to the
331                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332                     # recover, and the next same-md5 submit re-binds the truthful path via
333                     # add_video's upsert.
334                     video_path = str(report.get("video_uri") or "") or _first_object_under(
335                         f"{output_root}/videos/{video_id}/", sc
336                     )
337                 play_wm, cand_wm = db.segment_watermarks(video_id)
338                 db.add_video(video_id, video_path, "processed", [])
```

```
339                 n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340                 if n <= 0:
341                     return None
342                 segments = [
343                     s
344                     for s in db.query_play_segments(video_id, {})
345                     if int(s.get("id", 0)) > play_wm
346                 ]
347                 _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348                 if not segments:
349                     return None
```

9. user (2026-07-06T17:27:07.355Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18
19      import json
20      import logging
21      import os
22      import threading
23      import traceback
24      from concurrent.futures import ThreadPoolExecutor
25      from typing import Any, Dict, Optional
26
27      logger = logging.getLogger(__name__)
28
29      # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30      # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31      # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32      # already parallelize their AI handoff internally via their own pools.
33      # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34      # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35      # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36      # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37      # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38      # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39      #
40      # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41      # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42      # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43      #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
```

```
    #   - detector timeout_sec kills hung WSL/GPU calls
    #   - Cloud Run poll has a max_wait_sec bound
    #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
    # A future slice should add a per-job wall-clock timeout (the executor itself
    # cannot enforce timeouts on the Thread).
    _job_executor: Optional[ThreadPoolExecutor] = None
    # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
    # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
    # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
    _drain_max_workers: int = 1


    def _cloud_dispatch_possible(config: Any = None) -> bool:
        """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
        ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
        wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
        orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
        not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
        dep = getattr(config, "deployment", None)
        if getattr(dep, "compute_target", "") == "cloud_run":
            return True
        if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
            stor = getattr(config, "storage", None)
            if (getattr(stor, "output_root", "") or "").strip():
                return True
        return False


    def _resolve_drain_workers(config: Any = None) -> int:
        """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
        may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
        OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
        stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
        single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
        A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
        (tests / non-server) ? behaviour unchanged."""
        if _cloud_dispatch_possible(config):
            dep = getattr(config, "deployment", None)
            return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
        return 1


    def configure_drain(config: Any = None) -> None:
        """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
        server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
        global _drain_max_workers
        _drain_max_workers = _resolve_drain_workers(config)


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
```

```
95          ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96          global _job_executor
97          if _job_executor is None:
98              _job_executor = ThreadPoolExecutor(
99                  max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139             # the job row for analytics). NULL ? server default = pre-picker behaviour.
140             compute=job.get("compute"),
141             force=bool(params.get("force")),
142         )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
```

```
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:
162             if is_compile:
163                 result = _main._execute_compile(
164                     config,
165                     db,
166                     job,
167                     progress=lambda stage: db.set_job_progress(job_id, stage),
168                 )
169             else:
170                 req = _main._job_to_request(job)
171                 result = _main._execute_processing(
172                     config,
173                     db,
174                     req,
175                     progress=lambda stage: db.set_job_progress(job_id, stage),
176                 )
177             if result.get("video_id"):
178                 db.set_job_video_id(job_id, result["video_id"])
179             db.mark_job_done(job_id, result)
180             if not is_compile:
181                 _maybe_record_job_cost(
182                     job, result, config, db
183                 )  # M8 cost ledger (default-OFF; process jobs only)
184                 _maybe_run_flywheel(
185                     job, result, config
186                 )  # M11 data-flywheel (process jobs only)
187         except _main.JobWaiting as w:
188             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190             db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191         except Exception as e:
192             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193             db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196     def _maybe_record_job_cost(
197         job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198     ) -> None:
199         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
202         persist it.
203
204         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
205         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
206         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
207         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
```

```
        With the placeholder pricebook the cost is 0 regardless.

        Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
        claimed row (carries ``owner_id``); ``result`` is the pipeline output."""

        billing_cfg = getattr(config, "billing", None)
        if not billing_cfg or not getattr(billing_cfg, "enabled", False):
            return
        try:
            usage = (result or {}).get("usage") or {}
            db.record_job_cost(
                job["id"],
                usage=usage,
                pricebook_version=billing_cfg.pricebook_version,
                gpu_type=usage.get("gpu_type"),
                compute_backend=(result or {}).get("compute_backend"),
                owner_id=job.get("owner_id"),
                margin=billing_cfg.margin,
            )
            logger.info(
                "Recorded cost for job %s (pricebook=%s).",
                job["id"],
                billing_cfg.pricebook_version,
            )
        except Exception as e:  # never wedge a completed job on bookkeeping
            logger.warning(
                "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
            )


    def _maybe_run_flywheel(
        job: Dict[str, Any], result: Dict[str, Any], config: Any
    ) -> None:
        """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
        ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
        (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
        land), so this captures **nothing** in production. When activated it captures the
consented job's
        artifacts into the per-video workspace (? later labeling ? the golden training set).

        Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
        row (carries ``owner_id``); ``result`` is the pipeline output."""

        flywheel_cfg = getattr(config, "flywheel", None)
        if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
            return
        try:
            from backend import flywheel

            flywheel.run_for_job(job, result, config)
        except Exception as e:  # never wedge a completed job on the flywheel
            logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)


    def _drain_due_jobs(config: Any, db: Any) -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
```

```
picked
266         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267         a single-worker box keeps making progress with no central timer (the DB is the
clock).
268         Returns the number of jobs that reached a terminal/parked state this tick.
269         """
270         import backend.main as _main
271
272         ran = 0
273         while True:
274             job = db.claim_due_job()
275             if job is None:
276                 break
277             ran += 1
278             _main._run_claimed_job(job, config, db)
279         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281         # on the next submit/drain. Best-effort ? never raises into the worker.
282         _main._schedule_next_drain_if_waiting(config, db)
283         return ran
284
285
286     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288         self-scheduled job resumes with no further client submit. The drain itself re-checks
289         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:
293             delay = db.seconds_until_next_due()
294         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
295             logger.error(f"Could not compute next-due delay: {e}")
296             return
297         if delay is None:
298             return  # nothing waiting
299         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
307         import backend.main as _main
308
309         timer = threading.Timer(
310             delay_sec,
311             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312         )
313         timer.daemon = True
314         timer.start()
315
316
317     #: A parked job further out than this is re-checked by a capped wake rather than one
long
318     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319     _MAX_DRAIN_WAKE_SEC = 60.0
```

```
320
321
322     #
---------------------------------------------------------------------------------------------
323     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
324     # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
325     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
326     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
327     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
328     # re-dispatching (no double GPU spend).
329
330     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331     _RESUME_POLL_EVERY_SEC = 15.0
332
333
334     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
337         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
338         thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
339         NEVER re-dispatched from here. Returns the number of resumes started.
340
341         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
342         exist ? the rows get today's honest terminal failure instead."""
343         rows = db.interrupted_remote_process_jobs()
344         if not rows:
345             return 0
346         if not _cloud_dispatch_possible(config):
347             for job in rows:
348                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349             logger.warning(
350                 "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                 "dispatch to Cloud Run, so no remote worker can be in flight.",
352                 len(rows),
353             )
354             return 0
355         # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
356         # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
357         # default target itself is cloud_run (on a default-local server with cloud-override
358         # capability, a NULL row ran in-process ? no remote worker exists to resume).
359         default_is_cloud = (
360             getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
361         )
362         started = 0
363         for job in rows:
364             # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
365             # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
366             requested = (str(job.get("compute") or "")).strip().lower()
367             if requested != "cloud" and not default_is_cloud:
368                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
369                 continue
370             t = threading.Thread(
```

```
                target=_resume_one_remote_job,
                args=(config, db, dict(job)),
                daemon=True,
                name=f"resume-{str(job['id'])[:8]}",
            )
            t.start()
            started += 1
        if started:
            logger.warning(
                "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
                started,
            )
        return started


    def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
        """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
        ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
        fail the job HONESTLY from what the bucket says. Never dispatches."""
        from backend import orchestration, storage
        from backend.infrastructure.execution_policy import (
            ExecutionPolicy,
            PolicyTimeout,
            poll_until,
        )

        job_id = str(job["id"])
        video_id = str(job["video_id"])
        try:
            raw_params = job.get("params")
            params = (
                raw_params
                if isinstance(raw_params, dict)
                else json.loads(raw_params or "{}")
            )
            if params.get("force"):
                # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
                # but a forced job must NEVER complete from the bucket cache ? force
deliberately
                # bypasses it, and nothing proves the report belongs to the forced run
rather
                # than an older completed one.
                db.mark_job_failed(
                    job_id,
                    "control plane restarted during a forced reprocess; the cached result "
                    "cannot be trusted for force=true ? re-submit with force",
                )
                return
            if (str(job.get("compute") or "")).strip().lower() == "local":
                # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
                # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
                db.mark_job_failed(job_id, "interrupted by server restart")
                return
            sc = orchestration._storage_settings(config)
            output_root = ((sc or {}).get("output_root") or "").rstrip("/")
            if not sc or not output_root:
                db.mark_job_failed(
                    job_id,
                    "interrupted by server restart (no output bucket configured to recover
```

```
from)",
                )
                return
            report_uri = f"{output_root}/outputs/{video_id}/report.json"
            max_wait = float(
                getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
                or 3900.0
            )
            policy = ExecutionPolicy(
                poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
                op_timeout_sec=120.0,
                max_wait_sec=max_wait,
            )
            db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
            try:
                poll_until(
                    lambda: True if storage.exists(report_uri, config=sc) else None,
                    policy,
                    label=f"resume-{job_id[:8]}",
                    logger=logger,
                    # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
                    on_tick=lambda elapsed: db.set_job_progress(
                        job_id,
                        "recovering: waiting for the in-flight cloud run ? "
                        f"{int(elapsed) // 60} min",
                    ),
                )
            except PolicyTimeout:
                db.mark_job_failed(
                    job_id,
                    "control plane restarted mid-run and the cloud worker's result never "
                    "appeared within the poll budget; re-submit the video (a finished result
"
                    "re-hydrates instantly)",
                )
                return
            payload = orchestration.cached_process_result(
                db, config, video_id, str(job.get("video_path") or "")
            )
            if payload is not None:
                db.mark_job_done(job_id, payload)
                logger.warning(
                    "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
                    job_id,
                    video_id,
                )
                return
            # The report exists but did not realize into a usable result ? surface its
verdict.
            status = "unreadable"
            try:
                report = orchestration._read_bucket_json(report_uri, sc) or {}
                status = str(report.get("status") or "unreadable")
            except Exception:  # noqa: BLE001 ? the failure message is best-effort
                pass
            db.mark_job_failed(
                job_id,
                f"cloud worker finished without a usable result after a control-plane
restart "
                f"(report status: {status}); re-submit the video",
            )
        except Exception as e:  # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
```

```
486             logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487             try:
488                 db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489             except Exception:  # noqa: BLE001
490                 pass
491
```

10. assistant (2026-07-06T17:27:10.259Z)

Now let me look at the parallel yolo_hybrid segmenter (to check twin/base parity), the cloud_run consumer, and the existing gemini/motion Failure precedent.

11. user (2026-07-06T17:27:11.805Z)

```
160
161         def _select_for_handoff(
162             self,
163             windows: List[Dict[str, Any]],
164             window_stats: List[Dict[str, Any]],
165             idx_cfg: "IndexingConfig",
166         ) -> List[int]:
167             """Indices of the candidate windows that proceed to the AI handoff (and thus may
168             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
170             Persisted candidates are NOT affected ? they remain the full superset for
offline
171             re-scoring."""
172             return list(range(len(windows)))
173
174         def process_video(  # type: ignore[override]
175             self,
176             video_id: str,
177             video_path: str,
178             player_pool: Optional[List[str]] = None,
179             max_frames: Optional[int] = None,
180         ) -> "Union[List[Dict[str, Any]], Failure]":
181             # Partial adoption of the Result contract (the ABC declares Success|Failure).
182             # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
/
183             # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused with
a
184             # genuine 0-rally video (which still returns a bare ``[]``). Every caller
185             # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
186             # the bare-list success/empty paths are the documented incremental gap (CODE_MAP
gotchas).
187             label = self.display_label
188             # --- Sport profile + AI provider wiring (identical across segmenters) ---
189             sport_name = self.config.sport
190             try:
191                 sport_profile = sport_registry.get(sport_name)
192             except KeyError as e:
193                 logger.error(str(e))
194                 return []
195
196             idx_cfg = self.config.indexing
197             skip_ai = bool(idx_cfg.skip_ai_handoff)
198             provider_name = self.config.ai_provider
199             if skip_ai:
200                 model_name = "local-noai"
201                 logger.info(
202                     "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
203                     "be emitted as rallies; no %s handoff, no API key needed.",
204                     label,
205                     provider_name,
```

```
                )
            else:
                model_name = self.config.ai_model
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint

                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
                    )
                    return []
                try:
                    provider = provider_registry.get(
                        provider_name, api_key=api_key, model_name=model_name
                    )
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            try:
                runner, runner_params = self._resolve_runner(idx_cfg)
            except NotImplementedError as e:
                # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
                # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
                logger.error("%s: %s", label, e)
                return []

            velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
            merge_gap = idx_cfg.dead_time_merge_gap
            min_window_duration = idx_cfg.min_action_window_duration
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.max_window_duration
            window_min_split_gap = idx_cfg.window_min_split_gap
            window_hard_cap = idx_cfg.window_hard_cap
            window_inactivity_end = idx_cfg.window_inactivity_end
            inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
            inactivity_min_density = idx_cfg.inactivity_min_density
            inactivity_temporal_density = idx_cfg.inactivity_temporal_density
            window_blob_fallback = idx_cfg.window_blob_fallback
            window_blob_factor = idx_cfg.window_blob_factor
            handoff_padding = idx_cfg.ai_handoff_padding
            ai_max_workers = idx_cfg.ai_max_workers
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
```

```
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
                # failed/retryable instead of writing an empty-but-"successful" result.
                return Failure(
                    message=f"{label} detector environment not ready: {msg}",
                    is_recoverable=True,
                )
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join(output_base(self.config), self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
                # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
                # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
                # rather than a silent 0-segment "success" (the wf294 incident).
                return Failure(
                    message=(
                        f"{label} detector produced no trajectory (timed out or aborted) ? "
                        "detector failure, not a 0-rally video."
```

12. user (2026-07-06T17:27:12.481Z)

```
"""Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
(Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).

Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
(`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
can be consumed by the future fusion segmenter; this segmenter just orchestrates
chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.

Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
detection uses torchvision's permissively-licensed (BSD) detectors and the
association/tracking that YOLO.track() used to bundle in is provided by ByteTrack
via the standalone Apache-2.0 ``trackers`` package (D13/P12 migration, #386 ?
previously ``supervision.ByteTrack``, removed upstream in supervision 0.30.0; see
``person_detector.make_tracker`` for the behaviour-pinning details).

The registry key stays "yolo_hybrid" for back-compat with existing configs and the
DB `detector` tags, even though no YOLO is involved anymore.
"""

import concurrent.futures
import logging
import os
import time
from typing import Any, Dict, List, Tuple

```

```
27      from backend.pipeline.detectors.person_detector import PersonDetector
28      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
29      from backend.providers import provider_registry
30      from backend.sports import sport_registry
31      from backend.utils.paths import output_base
32      from backend.utils.video import extract_video_chunk, get_video_duration
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
39          seconds = max(0, int(seconds))
40          h, rem = divmod(seconds, 3600)
41          m, s = divmod(rem, 60)
42          return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45      class HybridYoloSegmenter(BasePlaySegmenter):
46          def __init__(self, db, config):
47              super().__init__(db, config)
48              # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
producer
49              # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
50              self.person = PersonDetector(config)
51
52          @property
53          def name(self) -> str:
54              return "yolo_hybrid"
55
56          @property
57          def description(self) -> str:
58              return (
59                  "Two-stage pipeline: torchvision person detection + ByteTrack for fast "
60                  "candidate filtering, then an AI provider for high-precision semantic
boundaries."
61              )
62
63          def _resolve_provider(self, video_path: str):
64              """Phase 0: resolve sport profile + AI provider (with API key).
65
66              Returns the configured provider instance, or ``None`` to signal that
67              ``process_video`` should early-return ``[]`` (bad sport, missing API key,
68              or unknown provider). Behaviour is identical to the inlined version.
69              """
70              # Get sport profile config
71              sport_name = self.config.sport
72              try:
73                  sport_registry.get(sport_name)
74              except KeyError as e:
75                  logger.error(str(e))
76                  return None
77
78              # Get AI provider and model config
79              provider_name = self.config.ai_provider
80              model_name = self.config.ai_model
81
82              # Get appropriate API key based on the provider
83              api_key_env_var = f"{provider_name.upper()}_API_KEY"
84              api_key = os.environ.get(api_key_env_var)
85              if not api_key:
86                  from backend.pipeline.segmenters._keyhint import api_key_hint
87
88                  logger.error(
89                      f"{api_key_env_var} environment variable is not set! "
```

```
                    f"To run the {provider_name} segmenter, set your API key. "
                    + api_key_hint(api_key_env_var)
                )
                return None

            try:
                provider = provider_registry.get(
                    provider_name, api_key=api_key, model_name=model_name
                )
            except KeyError as e:
                logger.error(str(e))
                return None

            logger.info(
                f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
            )
            return provider

        def _collect_candidate_windows(
            self,
            video_id: str,
            video_path: str,
            video_duration: float,
            idx_cfg: Dict[str, Any],
            chunk_duration: float,
            chunk_overlap: float,
            velocity_thresh: float,
            merge_gap: float,
            frame_skip: int,
            chunks_dir: str,
        ) -> List[Dict[str, Any]]:
            """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
            sequential), then dedup-merge overlapping windows across chunks. Verbatim
move."""
            step = chunk_duration - chunk_overlap
            chunk_starts = []
            t = 0.0
            while t < video_duration:
                chunk_starts.append(t)
                t += step

            num_chunks = len(chunk_starts)
            logger.info(
                f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks."
            )

            all_candidate_windows = []

            t_start = time.time()
            prefetch_workers = idx_cfg.yolo_prefetch_workers

            if num_chunks > 1 and prefetch_workers > 0:
                # --- PIPELINED PROCESSING ---
                # GPU inference must stay sequential (single GPU), but ffmpeg chunk
                # extraction is pure I/O. We pre-extract upcoming chunks in background
                # threads so extraction of chunk N+1 overlaps with inference on chunk N.
                logger.info(
                    f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)..."
                )

                extract_executor = concurrent.futures.ThreadPoolExecutor(
                    max_workers=prefetch_workers
```

```
                )

                def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
                    chunk_end = min(cs + chunk_duration, video_duration)
                    actual_dur = chunk_end - cs
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
                    success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
                    return (ci, cs, chunk_path, success)

                # Submit all extractions upfront ? they'll queue and run as workers free up
                extract_futures = [
                    extract_executor.submit(_extract_chunk, ci, cs)
                    for ci, cs in enumerate(chunk_starts)
                ]

                # Process results in order: wait for extraction, then run inference on GPU
                for future in extract_futures:
                    ci, chunk_start, chunk_path, success = future.result()
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    logger.info(
                        f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
                    )

                    if not success:
                        logger.warning(f"Chunk {ci + 1} extraction failed. Skipping.")
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path,
                        chunk_start,
                        velocity_thresh,
                        merge_gap,
                        frame_skip=frame_skip,
                        chunk_index=ci,
                    )
                    logger.info(
                        f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
                    )
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                extract_executor.shutdown(wait=False)
            else:
                # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
                for ci, chunk_start in enumerate(chunk_starts):
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_dur = chunk_end - chunk_start
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    logger.info(
                        f"Processing Chunk {ci + 1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)..."
                    )
                    success = extract_video_chunk(
                        video_path, chunk_start, actual_dur, chunk_path
                    )
                    if not success:
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
```

```
                        chunk_path,
                        chunk_start,
                        velocity_thresh,
                        merge_gap,
                        frame_skip=frame_skip,
                        chunk_index=ci,
                    )
                    logger.info(
                        f"Found {len(windows)} candidate action windows in chunk {ci + 1}."
                    )
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

            # Deduplicate overlapping candidate windows across chunks (chunks overlap by
            # chunk_overlap, so the same rally can surface in two adjacent chunks).
            def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
                fa, fb = a["active_frame_count"], b["active_frame_count"]
                total = fa + fb or 1
                return {
                    "start": a["start"],
                    "end": max(a["end"], b["end"]),
                    "active_frame_count": fa + fb,
                    "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
                    # Frame-weighted mean so longer sub-windows dominate proportionally.
                    "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb)
                    / total,
                    "track_id_count": max(a["track_id_count"], b["track_id_count"]),
                    "chunk_index": a["chunk_index"],
                }

            if all_candidate_windows:
                all_candidate_windows.sort(key=lambda w: w["start"])
                merged_windows = [all_candidate_windows[0]]
                for current in all_candidate_windows[1:]:
                    last = merged_windows[-1]
                    if current["start"] <= last["end"]:  # Overlap
                        merged_windows[-1] = _merge_stats(last, current)
                    else:
                        merged_windows.append(current)
                all_candidate_windows = merged_windows

            logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
            logger.info(f"Total candidate windows: {len(all_candidate_windows)}")

            return all_candidate_windows

        def _persist_candidates(
            self,
            video_id: str,
            all_candidate_windows: List[Dict[str, Any]],
            detection_params: Dict[str, Any],
            log_candidates: bool,
            store_candidates: bool,
        ) -> List[Any]:
            """Phase 2b: per-rally console log + persist raw candidates for the fine-tuning
            dataset. Returns window-index -> candidate_id (None where not stored). Verbatim
move."""
            # Per-rally console log (final, full-video timestamps) for video cross-
verification.
            if log_candidates:
                for w in all_candidate_windows:
```

```
277                     logger.info(
278                         f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
279                         f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
280                         f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
281                         f"tracks={w['track_id_count']}"
282                     )
283 
284             # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
285             # so confirmed AI segments can be linked back in Phase 4.
286             candidate_ids = [None] * len(all_candidate_windows)
287             if store_candidates:
288                 for i, w in enumerate(all_candidate_windows):
289                     stats = {
290                         "peak_velocity": w["peak_velocity"],
291                         "mean_velocity": w["mean_velocity"],
292                         "active_frame_count": w["active_frame_count"],
293                         "track_id_count": w["track_id_count"],
294                     }
295                     # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
296                     confidence = min(1.0, w["peak_velocity"])
297                     candidate_ids[i] = self.db.add_candidate_segment(
298                         video_id,
299                         detector="yolo_hybrid",
300                         start_time=w["start"],
301                         end_time=w["end"],
302                         chunk_index=w["chunk_index"],
303                         confidence=confidence,
304                         stats=stats,
305                         detection_params=detection_params,
306                     )
307 
308             return candidate_ids
309 
310         def _run_ai_handoff(
311             self,
312             video_id: str,
313             video_path: str,
314             video_duration: float,
315             all_candidate_windows: List[Dict[str, Any]],
316             candidate_ids: List[Any],
317             provider,
318             sport_profile,
319             provider_name: str,
320             chunk_duration: float,
321             handoff_padding: float,
322             ai_max_workers: int,
323         ) -> List[dict]:
324             """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
325             the provider for semantic boundaries, in parallel. Verbatim move."""
326             gemini_segments = []
327             gemini_dir = os.path.join(
328                 output_base(self.config), f"chunks_{provider_name}_handoff"
329             )
330             os.makedirs(gemini_dir, exist_ok=True)
331 
332             logger.info(
333                 f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
334             )
335 
336             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
337                 w_start = max(0, window["start"] - handoff_padding)
```

```
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start

                w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
                # Re-encode short handoff clips to avoid keyframe-boundary drift
                success = extract_video_chunk(
                    video_path, w_start, w_dur, w_path, reencode=True
                )
                if not success:
                    return []

                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, metrics = provider.analyze_video(
                    w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
                )
                # B2: a provider failure returns [] with metrics["error"] set ? that is
                # "we don't know", NOT "no rallies in this window". Surface it loudly so a
                # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
                if metrics.get("error"):
                    logger.warning(
                        "[Handoff %d] provider error ? window %d skipped: %s",
                        wi + 1,
                        wi,
                        metrics["error"],
                    )

                # Coordinate mapping ? shift local timestamps back to full-video time
                valid_segments = []
                for segment in segments:
                    try:
                        segment["start_time"] = float(segment["start_time"]) + w_start
                        segment["end_time"] = float(segment["end_time"]) + w_start
                        # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
                        segment["_candidate_id"] = candidate_ids[wi]
                        valid_segments.append(segment)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning(
                            f"Dropping segment with unparseable timestamps: {segment} ? {e}"
                        )

                try:
                    os.remove(w_path)
                except OSError:
                    pass

                return valid_segments

            with concurrent.futures.ThreadPoolExecutor(
                max_workers=ai_max_workers
            ) as executor:
                futures = [
                    executor.submit(process_candidate_window, wi, w)
                    for wi, w in enumerate(all_candidate_windows)
                ]
                # Iterate in submission order to preserve temporal ordering
                for future in futures:
                    gemini_segments.extend(future.result())

            try:
                os.rmdir(gemini_dir)
            except OSError:
                pass

```

```
            logger.info(
                f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
            )
            return gemini_segments

        def _populate_db(
            self, video_id: str, gemini_segments: List[dict], model_name: str
        ) -> List[Dict[str, Any]]:
            """Phase 4: DB Population ? write confirmed AI segments, link them back to their
            source candidate, and build the returned segments_data. Verbatim move."""
            segments_data = []
            for segment in gemini_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start

                    shot_count = segment.get("shuttle_exchange_count")
                    if shot_count is None:
                        shot_count = max(3, int(dur / 0.8))
                    else:
                        try:
                            shot_count = int(shot_count)
                        except (ValueError, TypeError):
                            shot_count = max(3, int(dur / 0.8))

                    metadata = {
                        "shot_count": shot_count,
                        "shot_count_confidence": segment.get(
                            "shot_count_confidence", "Unknown"
                        ),
                        "detector": f"yolo-hybrid-{model_name}",
                    }

                    segment_id = self.db.add_play_segment(
                        video_id, start, end, None, None, metadata
                    )
                    if segment_id:
                        # Link back to the candidate that produced this segment. A candidate
                        # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
                        # the last one ? the link table records "this candidate was
confirmed".
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append(
                            {
                                "id": segment_id,
                                "start_time": start,
                                "end_time": end,
                                "duration": dur,
                                "metadata": metadata,
                            }
                        )
                except Exception as e:
                    logger.warning(f"Skipping invalid segment struct: {e}")

            return segments_data

        def process_video(
            self,
            video_id: str,
            video_path: str,
```

```
            player_pool: List[str] = None,
            max_frames: int = None,
        ) -> List[Dict[str, Any]]:
            self.person.lazy_load_model()

            # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
            provider = self._resolve_provider(video_path)
            if provider is None:
                return []

            # Re-read the config values needed downstream (pure deterministic reads).
            idx_cfg = self.config.indexing
            sport_profile = sport_registry.get(self.config.sport)
            provider_name = self.config.ai_provider
            model_name = self.config.ai_model

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []

            chunk_duration = idx_cfg.chunk_duration
            chunk_overlap = idx_cfg.chunk_overlap
            velocity_thresh = (
                idx_cfg.yolo_velocity_threshold
            )  # Fraction of frame width per second
            merge_gap = idx_cfg.dead_time_merge_gap
            frame_skip = idx_cfg.yolo_frame_skip
            tracker_config = idx_cfg.yolo_tracker
            handoff_padding = idx_cfg.ai_handoff_padding
            ai_max_workers = idx_cfg.ai_max_workers
            log_candidates = idx_cfg.log_yolo_candidates
            store_candidates = idx_cfg.store_yolo_candidates

            # Snapshot of the params that produced these detections ? stored alongside each
            # candidate so a fine-tuning run can reproduce / correlate against them.
            detection_params = {
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "frame_skip": frame_skip,
                "tracker": tracker_config,
                "detector_model": idx_cfg.detector_model,
                "detector_score_threshold": idx_cfg.detector_score_threshold,
                "min_action_window_duration": idx_cfg.min_action_window_duration,
            }

            chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
            os.makedirs(chunks_dir, exist_ok=True)

            # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
            all_candidate_windows = self._collect_candidate_windows(
                video_id,
                video_path,
                video_duration,
                idx_cfg,
                chunk_duration,
                chunk_overlap,
                velocity_thresh,
                merge_gap,
                frame_skip,
                chunks_dir,
            )

            # Phase 2b: log + persist raw candidates (window index -> candidate_id).
```

```
527             candidate_ids = self._persist_candidates(
528                 video_id,
529                 all_candidate_windows,
530                 detection_params,
531                 log_candidates,
532                 store_candidates,
533             )
534 
535             try:
536                 os.rmdir(chunks_dir)
537             except Exception:
538                 pass
539 
540             # Phase 3: AI Provider Handoff
541             gemini_segments = self._run_ai_handoff(
542                 video_id,
543                 video_path,
544                 video_duration,
545                 all_candidate_windows,
546                 candidate_ids,
547                 provider,
548                 sport_profile,
549                 provider_name,
550                 chunk_duration,
551                 handoff_padding,
552                 ai_max_workers,
553             )
554 
555             # Phase 4: DB Population
556             return self._populate_db(video_id, gemini_segments, model_name)
557 
558 
559     # Register the segmenter
560     segmenter_registry.register(HybridYoloSegmenter)
561 
```

13. assistant (2026-07-06T17:27:19.248Z)

Now let me check the cloud_run consumer and main.py's Failure branch, plus how the tracknet subclass and base relate. Let me also examine the `_serialize_and_push_outputs` in the failure path more carefully ? specifically what happens when `scratch` may not be defined yet, and the `main.py` isinstance branch.

14. user (2026-07-06T17:27:20.818Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3 
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
```

```
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22      owner-verified on the M7 real run.
23      """
24
25      from __future__ import annotations
26
27      import json
28      import logging
29      import os
30      import tempfile
31      import uuid
32      from abc import ABC, abstractmethod
33      from typing import Any, Callable, List, Optional
34
35      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36      from backend.infrastructure.execution_policy import (
37          DEFAULT_POLICY,
38          ExecutionPolicy,
39          PolicyTimeout,
40          poll_until,
41      )
42      from backend.storage import fetch, get_storage, parse_uri
43
44      LOG = logging.getLogger("compute.cloud_run")
45
46      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
47      # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
48      _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
49          "deployment.compute_target=local_gpu",
50          "indexing.detector_impl=native",
51      )
52
53
54      class CloudRunJobClient(ABC):
55          """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
56
57          Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
58          overrides the job's container ``args`` for this execution and starts it."""
59
60          @abstractmethod
61          def run_job(
62              self,
63              job_name: str,
64              argv: List[str],
65              *,
66              region: str,
67              project: Optional[str] = None,
```

```
68          ) -> str:
69              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
70              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
71
72          @abstractmethod
73          def cancel_execution(self, execution: str) -> None:
74              """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
75              times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
76              May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77              NEVER masked by a cancel error)."""
78
79
80      class GoogleCloudRunJobClient(CloudRunJobClient):
81          """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84          def run_job(
85              self,
86              job_name: str,
87              argv: List[str],
88              *,
89              region: str,
90              project: Optional[str] = None,
91          ) -> str:
92              # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93              # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94              # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95              if not project:
96                  raise RuntimeError(
97                      "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                  )
100             try:
101                 from google.cloud import run_v2  # type: ignore[attr-defined]
102             except Exception as exc:  # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             )  # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]
115             )
116             op = client.run_job(
117                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
118             )
119             return (
```

```
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
```

```
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._read_marker(done_uri),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
```

```
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
                )
                return late
            try:
                self._client.cancel_execution(execution)
            except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
                cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
                LOG.warning(
                    "cloud-run: best-effort cancel of execution=%s failed: %s",
                    execution,
                    cancel_exc,
                    exc_info=True,
                )
            else:
                cancel_note = "cancel requested"
                LOG.info("cloud-run: requested cancel of execution=%s", execution)
            raise PolicyTimeout(
                f"{timeout_exc} [cloud-run job={self._job_name} "
                f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
                "state in the GCP console]"
```

```
293             ) from timeout_exc
294
295         # --- storage-backed completion signal (bucket-poll) ---
296         def _read_marker(self, done_uri: str) -> Optional[dict]:
297             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
298             ref = parse_uri(done_uri)
299             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
300                 return None
301             return (
302                 self._read_json(done_uri) or {}
303             )  # empty dict still terminates the poll (present == done)
304
305         def _read_json(self, uri: str) -> dict:
306             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
307             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
308             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
309             try:
310                 ref = parse_uri(uri)
311                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
312                     dst = os.path.join(td, ref.name or "obj.json")
313                     local = fetch(uri, dst, config=self._storage_config)
314                     with open(local, "r", encoding="utf-8") as f:
315                         data: Any = json.load(f)
316                 return data if isinstance(data, dict) else {}
317             except Exception:
318                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
319                 return {}
320
```

15. user (2026-07-06T17:27:21.434Z)

```
1530                     print("\n" + "=" * 40)
1531                     print("VALIDATION SUMMARY")
1532                     print("=" * 40)
1533                     for r in val_results:
1534                         status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535                         print(f"{status_symbol} {r.validator_name}: {r.message}")
1536                     print("=" * 40 + "\n")
1537
1538                     if failures:
1539                         print("[FAIL] Video failed checks. Indexing halted.")
1540                         return
1541
1542                     # Index
1543                     segmenter_cls = segmenter_registry.get(seg_name)
1544                     if not segmenter_cls:
1545                         print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546                         return
1547
1548                     print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549                     segmenter_actual = seg_name
1550                     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551                     segmenter = segmenter_cls(db_instance, global_config)
1552                     result = segmenter.process_video(
1553                         video_id, local_video, max_frames=args.max_frames
1554                     )
1555                     segmentation_ran = True
1556
1557                     if isinstance(result, Failure):
1558                         print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559                         print("[FAIL] Indexing halted for video.")
```

```
1560                        segments = []
1561                    else:
1562                        segments = result.value if hasattr(result, "value") else result
1563                        print(
1564                            f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565                        )
1566                    # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567                    _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568                finally:
1569                    # Always emit the per-video observability report. Never raises.
1570                    paths = emit_indexer_report(
1571                        db=db_instance,
1572                        video_id=video_id,
1573                        # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                        # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                        # in DB provenance via db.add_video.
1576                        video_path=local_video,
1577                        config=_report_config_for_input(global_config, args.video_path),
1578                        val_results=val_results,
1579                        val_context=val_context,
1580                        segmenter_requested=args.segmenter,
1581                        segmenter_actual=segmenter_actual,
1582                        was_reingest=was_reingest,
1583                        segmentation_ran=segmentation_ran,
1584                    )
1585                    if paths:
1586                        print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587                        print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588                        print(f"[REPORT] Machine JSON     : {paths.get('json')}")
1589            finally:
1590                storage.cleanup_acquired_local_input(
1591                    args.video_path, local_video, storage_cfg
1592                )
1593            return
1594
1595        # Start Local Server Mode
1596        if args.app or args.server or not args.video_path:
1597            if args.app:
1598                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
```

16. assistant (2026-07-06T17:27:28.465Z)

Now I have a good picture. Let me examine the base segmenter class to confirm the ABC contract, and check how `tracknet_hybrid` and `fusion` subclass relate to the base `process_video`. Let me also check the existing gemini/motion Failure precedent for `is_recoverable` convention.

17. user (2026-07-06T17:27:30.560Z)

```
1       from abc import ABC, abstractmethod
2       from typing import Any, Dict, List, Optional, Type
3
4       from backend.infrastructure.database import Database
5       from backend.results import (
6           Result,  # new standardized contract (Result[Value] == Success[Value] | Failure)
7       )
8
9
10      def _normalize_config(config: Any) -> Any:
```

```
        """Normalize a plain-dict config into an AppConfig so segmenters can use typed
        attribute access consistently. Production always passes AppConfig; tests may
        pass plain dicts ? this bridge keeps both paths working."""
        if isinstance(config, dict):
            from backend.config.models import AppConfig

            try:
                return AppConfig(**config)
            except Exception:
                return (
                    config  # invalid config ? let the segmenter's own validation reject it
                )
        return config


    class BasePlaySegmenter(ABC):
        def __init__(self, db: Database, config: Dict[str, Any]):
            self.db = db
            self.config = _normalize_config(config)

        @property
        @abstractmethod
        def name(self) -> str:
            """Unique identifier for this segmenter."""
            pass

        @property
        @abstractmethod
        def description(self) -> str:
            """Human-readable explanation of this segmenter."""
            pass

        @abstractmethod
        def process_video(
            self,
            video_id: str,
            video_path: str,
            player_pool: List[str] = None,
            max_frames: int = None,
        ) -> "Result[List[Dict[str, Any]]]":
            """
            Processes a video to detect play segments, populates SQLite database,
            and returns a Result.

            Use isinstance(result, Failure) (or result.is_success()) to distinguish
            "no segments found" from "something went wrong" (missing key, healthcheck
            failure, etc.). This is the start of the standardized error/result contract.
            """
            pass


    class SegmenterRegistry:
        def __init__(self):
            self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

        def register(self, segmenter_cls: Type[BasePlaySegmenter]):
            """Registers a new segmenter class under its name property."""
            try:
                temp_inst = segmenter_cls(None, {})
                name = temp_inst.name
            except Exception as e:
                raise ValueError(
                    f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}"
                ) from e
```

```
            self._segmenters[name] = segmenter_cls

        def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
            return self._segmenters.get(name)

        def list_available(self) -> Dict[str, str]:
            """Returns a dict of registered segmenter names and their descriptions."""
            res = {}
            for name, cls in self._segmenters.items():
                try:
                    temp_inst = cls(None, {})
                    desc = temp_inst.description
                except Exception:
                    desc = "No description available."
                res[name] = desc
            return res


    segmenter_registry = SegmenterRegistry()

```

18. user (2026-07-06T17:27:30.860Z)

```
    """TrackNet-based hybrid play segmenter.

    Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
    in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
    tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
    purpose-built trajectory model localises it far more precisely, giving cleaner
    candidate rallies than person-velocity heuristics.

    The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
    lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
    TrackNet runner and identity.
    """

    from typing import Any, Dict, Tuple

    from backend.config.models import IndexingConfig
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter


    class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
        family = "tracknet"
        display_label = "TrackNet"

        @property
        def name(self) -> str:
            return "tracknet_hybrid"

        @property
        def description(self) -> str:
            return (
                "Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
                "candidate rallies + AI provider for high-precision semantic boundaries."
            )

        def _build_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
```

```
42              return TrackNetRunner(cfg), {
43                  "weights_path": cfg.weights_path,
44                  "queue_length": cfg.queue_length,
45              }
46
47
48      segmenter_registry.register(TrackNetHybridSegmenter)
49
```

19. assistant (2026-07-06T17:27:33.987Z)

The tracknet subclass only overrides `_build_runner`, not `process_video`, confirming base parity. Let me verify the fusion segmenter also doesn't override `process_video`, and check the gemini/motion Failure precedent for `is_recoverable` convention.

20. user (2026-07-06T17:27:36.730Z)

```
backend/pipeline/segmenters/base.py:44:    def process_video(
backend/pipeline/segmenters/gemini.py:620:    def process_video(
backend/pipeline/segmenters/motion.py:23:    def process_video(
backend/pipeline/segmenters/trajectory_hybrid.py:174:    def process_video(  # type:
ignore[override]
backend/pipeline/segmenters/yolo_hybrid.py:459:    def process_video(
```

21. user (2026-07-06T17:27:37.563Z)

```
backend\pipeline\segmenters\motion.py:39:            return Failure(
backend\pipeline\segmenters\motion.py-40-                message=f"Could not open video:
{video_path}", is_recoverable=False
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\motion.py-41-            )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\motion.py-42-
--
backend\pipeline\segmenters\trajectory_hybrid.py:278:            return Failure(
backend\pipeline\segmenters\trajectory_hybrid.py-279-                message=f"{label} detector
environment not ready: {msg}",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-280-
is_recoverable=True,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-281-            )
--
backend\pipeline\segmenters\trajectory_hybrid.py:296:            return Failure(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-297-                message=(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-298-                    f"{label}
detector produced no trajectory (timed out or aborted) ? "
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-299-                    "detector
failure, not a 0-rally video."
--
backend\pipeline\segmenters\gemini.py:100:            return Failure(str(e))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-101-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-102-        # Get AI provider and model config
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-103-        provider_name =
self.config.ai_provider
--
backend\pipeline\segmenters\gemini.py:116:            return Failure(msg)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
```

```
indexer\backend\pipeline\segmenters\gemini.py-117-
backend\pipeline\segmenters\gemini.py-118-        try:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-119-            provider = provider_registry.get(
--
backend\pipeline\segmenters\gemini.py:124:            return Failure(str(e))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-125-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-126-        return sport_profile, provider
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-127-
```

22. assistant (2026-07-06T17:27:44.178Z)

Let me check the fusion segmenter (referenced as a subclass of the base) and confirm it doesn't override `process_video`. Also let me look at the existing test file to understand `_make_runner` and `_OKResult` helpers so I can validate the test assertions.

23. user (2026-07-06T17:27:46.382Z)

```
__init__.py
__pycache__
_keyhint.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
---FUSION---
backend/pipeline/segmenters/fusion_hybrid.py
backend/pipeline/segmenters/tracknet_hybrid.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/pipeline/segmenters/wasb_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/segmenters/__init__.py
backend/pipeline/segmenters/__pycache__/fusion_hybrid.cpython-313.pyc
backend/pipeline/segmenters/__pycache__/tracknet_hybrid.cpython-313.pyc
backend/pipeline/segmenters/__pycache__/trajectory_hybrid.cpython-313.pyc
backend/pipeline/segmenters/__pycache__/wasb_hybrid.cpython-313.pyc
backend/pipeline/segmenters/__pycache__/yolo_hybrid.cpython-313.pyc
backend/pipeline/segmenters/__pycache__/__init__.cpython-313.pyc
```

24. user (2026-07-06T17:27:46.811Z)

```
1       """Tests for the TrackNetHybridSegmenter pipeline. The WSL/TensorFlow runner is
2       mocked out (TrackNetRunner), so these run anywhere ? they exercise Phase 1-4 glue:
3       healthcheck gating, candidate persistence, AI handoff, and DB population/linking."""
4
5       from unittest.mock import MagicMock, patch
6
7       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
8       from backend.results import Failure
9
10
11      def _window(start=1.0, end=4.0):
12          return {
13              "start": start,
14              "end": end,
15              "active_frame_count": 30,
16              "peak_velocity": 0.4,
```

```
            "mean_velocity": 0.2,
            "track_id_count": 1,
            "chunk_index": 0,
        }


    def _patched(fn):
        """Stack the patches every pipeline test needs. fps/width probing is stubbed
        so cv2 internals don't need mocking (covered separately).

        Patches are applied innermost-first, and mock.patch injects the innermost
        patch as the *first* positional arg ? so this order matches the signature
        (mock_runner_cls, mock_probe, mock_dur, mock_extract, mock_provider_registry,
        mock_env)."""
        fn = patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
        fn = patch.object(
            TrackNetHybridSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)
        )(fn)
        # Shared pipeline collaborators live in the trajectory_hybrid base module.
        fn = patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration")(fn)
        fn = patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk")(fn)
        fn = patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry")(fn)
        fn = patch("os.environ.get")(fn)
        return fn


    def _make_runner(windows=None, csv="out.csv", healthy=True):
        runner = MagicMock()
        runner.healthcheck.return_value = (
            healthy,
            "TF 2.17 GPUs 1" if healthy else "no GPU",
        )
        runner.run_predict.return_value = csv
        runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
        runner.trajectory_to_action_windows.return_value = (
            windows if windows is not None else [_window()]
        )
        return runner


    @_patched
    def test_pipeline_happy_path(
        mock_runner_cls,
        mock_probe,
        mock_dur,
        mock_extract,
        mock_provider_registry,
        mock_env,
    ):
        mock_env.return_value = "dummy_key"
        mock_dur.return_value = 30.0
        mock_extract.return_value = True
        mock_runner_cls.return_value = _make_runner()

        provider = MagicMock()
        provider.analyze_video.return_value = (
            [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}],
            {},
        )
        mock_provider_registry.get.return_value = provider

        db = MagicMock()
        db.add_candidate_segment.return_value = 55
        db.add_play_segment.return_value = 200

```

```
82          seg = TrackNetHybridSegmenter(db, {"indexing": {}})
83          res = seg.process_video("v1", "clip.mp4")
84
85          assert len(res) == 1
86          assert res[0]["id"] == 200
87          # Candidate persisted under the tracknet detector name...
88          _, kwargs = db.add_candidate_segment.call_args
89          assert kwargs["detector"] == "tracknet_hybrid"
90          # ...and the confirmed segment linked back to it.
91          db.link_candidate_to_segment.assert_called_once_with(55, 200)
92
93
94      @_patched
95      def test_pipeline_no_api_key_returns_empty(
96          mock_runner_cls,
97          mock_probe,
98          mock_dur,
99          mock_extract,
100         mock_provider_registry,
101         mock_env,
102     ):
103         mock_env.return_value = None
104         mock_dur.return_value = 30.0
105         mock_runner_cls.return_value = _make_runner()
106
107         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
108         assert seg.process_video("v1", "clip.mp4") == []
109
110
111     @_patched
112     def test_pipeline_healthcheck_failure_aborts(
113         mock_runner_cls,
114         mock_probe,
115         mock_dur,
116         mock_extract,
117         mock_provider_registry,
118         mock_env,
119     ):
120         mock_env.return_value = "dummy_key"
121         mock_dur.return_value = 30.0
122         mock_extract.return_value = True
123         mock_runner_cls.return_value = _make_runner(healthy=False)
124         mock_provider_registry.get.return_value = MagicMock()
125
```

25. assistant (2026-07-06T17:27:51.699Z)

Let me check the fusion_hybrid to confirm it doesn't override `process_video`, and the wasb_hybrid. Then I want to look at a subtle point: the healthcheck-not-ready path returns `Failure` ? but note there are OTHER early-return `[]` paths in `process_video` (no API key, bad sport, invalid duration, NotImplementedError). Those still return bare `[]`. Let me verify whether that's an inconsistency worth flagging by checking the no-api-key case: does that path represent a real failure that would now be masked as "success"?

26. user (2026-07-06T17:27:53.739Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
```

```
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28
29      from typing import Any, Dict, List, Optional, Tuple
30
31      from backend.config.models import FusionConfig, IndexingConfig
32      from backend.pipeline.detectors import fusion_features as ff
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors.base import DetectorRunner
35      from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36      from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
37      from backend.pipeline.segmenters.base import segmenter_registry
38      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
39
40
41      class FusionSegmenter(TrajectoryHybridSegmenter):
42          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
43
44          family = "fusion"
45          display_label = "Fusion"
46
47          def __init__(self, db: Any, config: Any):
48              super().__init__(db, config)
49              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
50              self._person: Optional[Any] = None
51              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
52              # over the whole trajectory once per candidate window (it depends only on
`points`).
53              self._axis_cache_key: Optional[int] = None
54              self._axis_cache: Optional[tuple] = None
55
56          @property
57          def name(self) -> str:
58              return "fusion_hybrid"
59
60          @property
```

```
        def description(self) -> str:
            return (
                "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
                "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and
the "
                "AI provider sets semantic boundaries."
            )

        def _build_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            if isinstance(idx_cfg, dict):
                idx_cfg = IndexingConfig(**idx_cfg)
            substrate = idx_cfg.fusion.substrate
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {
                "weights_path": wcfg.weights_path,
                "fusion_substrate": "wasb",
            }

        def _build_native_runner(
            self, idx_cfg: IndexingConfig
        ) -> Tuple[DetectorRunner, Dict[str, Any]]:
            # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
            # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
            substrate = idx_cfg.fusion.substrate
            if substrate != "wasb":
                raise NotImplementedError(
                    "detector_impl='native' supports the WASB substrate only ? set "
                    "indexing.fusion.substrate='wasb' (no native TrackNet runner yet)."
                )
            from backend.pipeline.detectors.native_wasb_runner import (
                NativeWasbConfig,
                NativeWasbRunner,
            )

            cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
            return NativeWasbRunner(cfg), {
                "weights_path": cfg.weights_path,
                "detector_impl": "native",
                "fusion_substrate": "wasb",
                "device": cfg.device,
            }

        def _enrich_candidate_stats(
            self,
            cw: Dict[str, Any],
            points: List[Any],
            fps: float,
            frame_width: float,
            video_path: str,
            idx_cfg: IndexingConfig,
        ) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(
                cw, points, fps, frame_width, video_path, idx_cfg
```

```
            )
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
            gated = rally_gate.gate_windows(
                points,
                [(cw["start"], cw["end"])],
                fps,
                min_crossings=0,
                estimate=self._axis_estimate(points),
            )
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.fusion
            if fcfg.cue_flags.get("person", False):
                stats.update(
                    self._person_features(cw, points, fps, frame_width, video_path, fcfg)
                )
            return stats

        # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
        # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
        # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
        # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
        # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
        # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
        # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.

        def _axis_estimate(self, points: List[Any]) -> tuple:
            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
            computed once per video, not once per candidate window (it depends only on
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(
            self,
            cw: Dict[str, Any],
            points: List[Any],
            fps: float,
            frame_width: float,
            video_path: str,
            fcfg: "FusionConfig",
        ) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
```

```
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector

            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.indexing.yolo_frame_skip
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"]
            # If we can't trust the frame dims, every normalised feature would silently
collapse
            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
            if fw <= 0 or fh <= 0:
                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

            poly = fcfg.court_polygon
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.net_axis, float(fcfg.net_position))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(
                det["frames"], shuttle_in, fw, fh, court_polygon=polygon, net=net
            )

        def _select_for_handoff(
            self,
            windows: List[Dict[str, Any]],
            window_stats: List[Dict[str, Any]],
            idx_cfg: IndexingConfig,
        ) -> List[int]:
            """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
            (0) keep every window ? identical to the substrate. Persisted candidates are
            untouched (full superset).

            NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
            thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
            where no segment clears ``min_crossings``), the gate falls back to the full
ungated
            set rather than emit a video with zero rallies. This makes Gate A/B strictly
            *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
            the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
            below it always wins on an otherwise-empty video. Observed on real footage
            (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
            gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
            fcfg = idx_cfg.fusion
            min_crossings = int(fcfg.min_crossings)
            min_players = int(fcfg.min_players)
            person_on = bool(fcfg.cue_flags.get("person", False))

            keep: List[int] = []
            for i, st in enumerate(window_stats):
                if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
                    continue
```

```
                if (
                    person_on
                    and min_players > 0
                    and st.get("players_in_court", 0.0) < min_players
                ):
                    continue
                keep.append(i)

            if not keep and window_stats:
                # Gating emptied a non-empty video ? fall back to the ungated set.
                return list(range(len(window_stats)))
            return keep


    segmenter_registry.register(FusionSegmenter)

```

27. user (2026-07-06T17:27:54.264Z)

```
      """WASB-based hybrid play segmenter.

      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
      badminton weights and runs today, so this is the live shuttle-motion substrate.

      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
      min_window_duration) are what get tuned against golden rally labels.

      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
      WASB runner and identity.
      """

      from typing import Any, Dict, Tuple

      from backend.config.models import IndexingConfig
      from backend.pipeline.detectors.base import DetectorRunner
      from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
      from backend.pipeline.segmenters.base import segmenter_registry
      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter


      class WasbHybridSegmenter(TrajectoryHybridSegmenter):
          family = "wasb"
          display_label = "WASB"

          @property
          def name(self) -> str:
              return "wasb_hybrid"

          @property
          def description(self) -> str:
              return (
                  "Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
                  "weights) for precise candidate rallies + AI provider for semantic
boundaries."
              )

          def _build_runner(
              self, idx_cfg: IndexingConfig
          ) -> Tuple[DetectorRunner, Dict[str, Any]]:
              cfg = WasbConfig.from_indexing_cfg(idx_cfg)
              return WasbRunner(cfg), {"weights_path": cfg.weights_path}
```

```
45
46          def _build_native_runner(
47              self, idx_cfg: IndexingConfig
48          ) -> Tuple[DetectorRunner, Dict[str, Any]]:
49              # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
50              # seam (imported only when detector_impl='native' is actually selected).
51              from backend.pipeline.detectors.native_wasb_runner import (
52                  NativeWasbConfig,
53                  NativeWasbRunner,
54              )
55
56              cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
57              return NativeWasbRunner(cfg), {
58                  "weights_path": cfg.weights_path,
59                  "detector_impl": "native",
60                  "device": cfg.device,
61              }
62
63
64      segmenter_registry.register(WasbHybridSegmenter)
65
```

28. assistant (2026-07-06T17:28:07.589Z)

Confirmed: base parity holds ? none of the three trajectory subclasses (wasb/tracknet/fusion) override `process_video`, so the fix applies to all three.

Now there is an important consistency question. The `process_video` in `trajectory_hybrid.py` has SEVERAL other early-return `[]` paths that represent genuine failures, NOT 0-rally videos:
- Line 194: bad sport ? `return []`
- Line 219: missing API key ? `return []`
- Line 226: unknown provider ? `return []`
- Line 231: invalid video duration ? `return []`
- Line 244: NotImplementedError (native runner unavailable for family) ? `return []`

Let me check whether these are pre-existing behavior or introduced/left inconsistent by this change. The missing-API-key case is notable: with `skip_ai_handoff=False` and no key, the segmenter "bails BEFORE detection" and returns `[]` ? which the worker will now mark as a 0-segment "success". That's exactly the class of silent-failure the wf294 fix targets. But is it in scope? The task explicitly scoped the fix to the TWO detector paths. Let me verify these `[]` returns are pre-existing (not touched by the diff) ? they are, per the diff. So this is a pre-existing gap, not a regression. I'll note it but weigh scope carefully.

Let me now look at the `_serialize_and_push_outputs` failure-path concern: in `_fail`, it's called with `scratch`, `args.out_uri`, `video_id`. But `_fail(2)` for validation failure and unknown-segmenter is called AFTER `video_id` and `scratch` are defined. However, `_fail` is also invoked for validation failure ? let me check the ordering carefully, and whether the report.json written by `_fail` could OVERWRITE a legitimate earlier success. Let me look at the zero-segments test and `_OKResult` helper.

29. user (2026-07-06T17:28:09.609Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
6       """
7
8       import json
9
10      import pytest
11
```

```
    from backend.config import load_config
    from backend.results import Failure
    from backend.worker import __main__ as wmain

    # ------------------------------------------------------------- pure helpers


    @pytest.mark.parametrize(
        "raw,expected",
        [
            ("true", True),
            ("False", False),
            ("3", 3),
            ("2.5", 2.5),
            ("wasb_hybrid", "wasb_hybrid"),
        ],
    )
    def test_coerce(raw, expected):
        assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


    def test_apply_overrides_on_typed_config():
        cfg = _fresh_config()
        wmain._apply_overrides(
            cfg,
            [
                "indexing.skip_ai_handoff=true",
                "indexing.min_segment_duration=1.5",
                "sport=tennis",
            ],
        )
        assert cfg.indexing.skip_ai_handoff is True
        assert cfg.indexing.min_segment_duration == 1.5
        assert cfg.sport == "tennis"


    def test_apply_overrides_rejects_bad_format():
        cfg = _fresh_config()
        with pytest.raises(ValueError):
            wmain._apply_overrides(cfg, ["no_equals_sign"])


    def test_write_intervals_csv(tmp_path):
        segs = [
            {
                "start_time": 2.0,
                "end_time": 5.0,
                "shot_count": 4,
                "shot_count_confidence": "LocalNoAI",
```

30. user (2026-07-06T17:28:10.263Z)

```

        def process_video(self, video_id, path, max_frames=None):
            return Failure(
                message="WASB detector produced no trajectory (timed out or aborted)",
                is_recoverable=True,
            )


    def test_setup_logging_levels():
        import logging

        wmain._setup_logging(False)
        assert logging.getLogger().level == logging.INFO
```

```
        wmain._setup_logging(True)
        assert logging.getLogger().level == logging.DEBUG


    def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
        # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"

        # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
        # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
        args = wmain.build_parser().parse_args(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
            ]
        )
        with caplog.at_level("WARNING", logger="backend.worker"):
            rc = wmain.cmd_infer(args)
        assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
        msgs = " ".join(r.getMessage() for r in caplog.records)
        assert "0 segments" in msgs and "vidZ" in msgs
        assert "detector_impl=native" in msgs  # surfaces the resolved detector
        assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
        # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
        # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
        report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
        assert report["segment_count"] == 0 and report["status"] == "success"


    def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
        tmp_path, monkeypatch
    ):
        """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
        as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
        status="failed" so the control plane / alpha-user surface can show an error /
retry."""
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
```

```
            lambda path, cfg: ([_OKResult()], {}),
        )
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"
        done = tmp_path / "bucket" / "_dispatch" / "tok.done"

        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(out),
                "--scratch",
                str(tmp_path / "s"),
                "--segmenter",
                "wasb_hybrid",
                "--done-uri",
                str(done),
            ]
        )
        # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
        assert rc == 3
        # report.json is written as an explicit FAILURE, not a false empty "success".
        report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
        assert report["status"] == "failed" and report["segment_count"] == 0
        assert report["segments"] == []
        # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
        marker = json.loads(done.read_text())
        assert marker["returncode"] == 3 and marker["status"] == "failed"
        assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"


    def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
        monkeypatch.setattr(
            wmain.validator_registry,
            "run_all_with_context",
            lambda path, cfg: ([_FailResult()], {}),
        )
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"X")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        rc = wmain.main(
            [
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
                str(tmp_path / "o"),
                "--config",
                str(cfg),
                "--scratch",
                str(tmp_path / "s"),
            ]
        )
        assert rc == 2


```

```
344     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
345
346
347     @pytest.mark.parametrize(
348         "fname,expected",
349         [
350             (
351                 "2026-06-22_GX020094.rallies.csv",
352                 "GX020094",
353             ),  # canonical dated label (DATA_IN_GCS §7.1)
354             ("GX020094.rallies.csv", "GX020094"),  # legacy bare label -> SAME clip stem
355             (
356                 "GX020094_proxy.rallies.csv",
357                 "GX020094_proxy",
358             ),  # _proxy is part of the name, kept
359             (
360                 "2026-06-21_mahadevpura_1.rallies.csv",
361                 "mahadevpura_1",
362             ),  # underscores in <name> survive
363             ("v1.csv", "v1"),  # bare .csv suffix
364             ("2026-06-22_GX020094.csv", "GX020094"),  # dated + bare .csv
365         ],
366     )
367     def test_label_clip_name_strips_date_prefix(fname, expected):
368         """A dated label and a bare label must both resolve to the bare videos/<name>
stem."""
369         assert wmain._label_clip_name(fname) == expected
```

31. assistant (2026-07-06T17:28:23.464Z)

Now let me look at the validation-failure test more closely. The new `_fail(2)` for validation also now writes a `report.json` with status="failed". But `test_cmd_infer_validation_failure_returns_2` only asserts `rc == 2` ? it doesn't assert that report.json was written. That's a test gap but minor. More importantly, let me verify a subtle behavioral concern: in the validation-failure path, `_fail(2)` calls `_serialize_and_push_outputs` which lists `os.listdir(out_local)`. But does the validation `_fail(2)` path get reached BEFORE the `try/finally` that emits the indexer report? Let me re-trace.

Looking at cmd_infer: the `_fail` calls at lines 312 and 320 (validation failure, unknown segmenter) are INSIDE the `try:` block (line 306), whose `finally:` calls `emit_indexer_report`. So `return _fail(2)` executes the finally. That's fine.

Now, a key question: the validation-failure path. When validation fails, `_fail(2)` now writes `report.json` with status="failed". Previously it did NOT write any report.json. Is there a consumer that now behaves differently? The `orchestration.cached_process_result` and `probe_process_cache` check `report.get("status") == "success"` ? a "failed" report returns None/miss ? dispatch normally. That's consistent.

Let me check a potential concurrency/idempotency issue: `probe_process_cache` returns True only for `status == "success"`. But there's an important edge ? for a video that LEGITIMATELY had 0 rallies (bare `[]` ? status="success", segment_count 0), `cached_process_result` at line 339-341: `n = _ingest_remote_segments(...)` and `if n <= 0: return None`. Let me read that tail.

32. user (2026-07-06T17:28:25.087Z)

```
348                 if not segments:
349                     return None
350                 logger.info(
351                     "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352                     n,
353                     video_id,
354                     outputs_uri,
```

```
355                 )
356                 return _cached_process_payload(video_id, [], segments, requested_compute)
357         except Exception:  # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358             logger.warning(
359                 "[API] submit-time cache check failed for %s ? dispatching normally",
360                 video_id,
361                 exc_info=True,
362             )
363             return None
364 
365 
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception:  # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372 
373 
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375 
376 
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
379         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
so
380         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
381         has the video's rows now (re-query). Non-md5 ids return False without touching the
382         bucket; every failure degrades to False (the read just reports what the DB has)."""
383         vid = (video_id or "").strip().lower()
384         if not _MD5_HEX_RE.fullmatch(vid):
385             return False
386         if not probe_process_cache(db, cfg, vid):
387             return False
```

33. assistant (2026-07-06T17:28:44.647Z)

This confirms an important pre-existing behavior that is NOT changed by this fix, but is relevant: a legitimately-0-rally video writes `status="success"` with `segment_count=0`. In `cached_process_result`, that success report leads to `_ingest_remote_segments` returning `n <= 0` ? `return None` (cache miss). So a genuine 0-rally video would be RE-DISPATCHED on every resubmit, never cache-served. That's a pre-existing quirk unrelated to this diff; I won't report it.

Now let me focus on the resume path ? `_resume_one_remote_job`. This is the important control-plane interaction for the wf294 fix. It polls for report.json to EXIST, then calls `cached_process_result`. For a detector-failure that now writes `status="failed"` report.json:
1. `poll_until` sees report.json exists ? returns.
2. `cached_process_result` ? report status != "success" ? returns None (line 326-327).
3. Falls to line 473-484: reads report, `status = "failed"`, calls `db.mark_job_failed(...)` with "report status: failed".

That is the CORRECT new behavior ? the job is now marked failed instead of hanging. Good. This is the key win.

Now let me verify the actual concern in the diff's `_fail` ? one thing stands out. In `_fail`, `_serialize_and_push_outputs` writes report.json THEN the done-marker is written. But note the ordering vs. the cloud dispatcher's poll. The cloud dispatcher (`CloudRunCompute.run_infer`) polls the **done-marker**, and once it sees the marker, reads `report.json`. In `_fail`, report.json is pushed FIRST (best-effort), THEN the marker. So when the dispatcher sees the marker, report.json already exists. Good ordering.

But `_resume_one_remote_job` polls for **report.json** existence (not the marker). In `_fail`,

report.json is written best-effort inside a try/except. If that push fails (exception caught, warning logged), the marker is STILL written but report.json is absent. Then the resume path would poll forever for a report.json that never arrives ? PolicyTimeout ? job failed after full budget. That's a degraded-but-safe outcome (still fails, just slowly). Pre-existing concern with best-effort semantics; acceptable.

Let me now check one subtle detail: in the SUCCESS path (line 389-392), the done-marker is written with `status` variable. When `segments` is a legit empty list, `status="success"`. Good. Let me verify there's no path where `status` stays "failed" erroneously in the success tail. At line 305, `status = "failed"` initially; line 330 sets `status = "success"` after the isinstance check passes. The `_serialize_and_push_outputs` at line 369 uses `status`. Since we only reach line 369 if we did NOT return via `_fail`, and status was set to "success" at line 330, this is correct ? UNLESS the segmenter returned a `Success` object. Let me check: line 329 `segments = result.value if hasattr(result, "value") else result`. If `process_video` returns a bare list (current reality), `segments = result`. Status="success". Fine.

Now let me verify the test for the done-marker in the zero-segments case. The `test_cmd_infer_zero_segments_is_logged_loudly` test does NOT pass `--done-uri`, so no marker. And the new detector-failure test DOES check the marker. Good coverage split. Let me check if there's an end-to-end test using the REAL trajectory-hybrid Failure (not just `_FailSeg`). Let me search.

34. user (2026-07-06T17:28:46.306Z)

```
Found 12 files
tests\test_worker.py
tests\test_tracknet_hybrid.py
tests\test_startup_checks.py
tests\test_native_wasb_runner.py
tests\test_wasb_runner.py
tests\test_tracknet_runner.py
tests\test_rally_seg_eval.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_stub_runner.py
tests\test_nightly_regression.py
tests\test_output_base.py
tests\test_fusion_hybrid.py
```

35. user (2026-07-06T17:28:47.510Z)

```
73:    out = tmp_path / "report.json"
187:    report = out / "outputs" / "vid123" / "report.json"
264:    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
265:    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
266:    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
267:    assert report["segment_count"] == 0 and report["status"] == "success"
274:    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
275:    status="failed" so the control plane / alpha-user surface can show an error / retry."""
306:    # report.json is written as an explicit FAILURE, not a false empty "success".
307:    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
308:    assert report["status"] == "failed" and report["segment_count"] == 0
312:    assert marker["returncode"] == 3 and marker["status"] == "failed"
```

36. assistant (2026-07-06T17:28:52.043Z)

Let me verify the roundtrip test at line 187 (`report = out / "outputs" / "vid123" / "report.json"`) to see if the success-path status is asserted there, and check whether any test verifies the report status="success" on a non-empty success. Let me also confirm there's no worker-level test that flows a REAL trajectory-hybrid Failure through cmd_infer.

37. user (2026-07-06T17:28:52.666Z)

```
150         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
151         monkeypatch.setattr(
152             wmain.validator_registry,
153             "run_all_with_context",
154             lambda path, cfg: ([_OKResult()], {}),
155         )
156         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
157         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
158
159
160     def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
161         video = tmp_path / "in.mp4"
162         video.write_bytes(b"FAKEVIDEO")
163         cfg = tmp_path / "config.json"
164         cfg.write_text("{}")
165         out = tmp_path / "bucket"
166         scratch = tmp_path / "scratch"
167
168         rc = wmain.main(
169             [
170                 "infer",
171                 "--video-uri",
172                 str(video),
173                 "--out-uri",
174                 str(out),
175                 "--config",
176                 str(cfg),
177                 "--scratch",
178                 str(scratch),
179                 "--segmenter",
180                 "wasb_hybrid",
181                 "--set",
182                 "indexing.skip_ai_handoff=true",
183             ]
184         )
185         assert rc == 0
186         intervals = out / "outputs" / "vid123" / "intervals.csv"
187         report = out / "outputs" / "vid123" / "report.json"
188         assert intervals.exists() and report.exists()
189         rows = intervals.read_text().splitlines()
190         assert len(rows) == 3  # header + 2 rallies
191         assert json.loads(report.read_text())["segment_count"] == 2
192
193
194     class _EmptySeg:
195         """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare []).."""
196
197         def __init__(self, db, config):
198             pass
199
200         def process_video(self, video_id, path, max_frames=None):
201             return []
202
203
204     class _FailSeg:
205         """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
206         trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
207
208         def __init__(self, db, config):
209             pass
```

38. assistant (2026-07-06T17:28:58.439Z)

Now I have a thorough understanding. Let me do a final verification of one concern: whether the

`test_cmd_infer_detector_failure...` test would actually catch a regression where `_fail` writes report.json but the marker path is broken, or vice versa. It asserts both. Good.

Let me now verify the mypy/ruff clean claims and that there isn't a stale comment issue in the diff. The task mentioned checking for the OLD "override-ignore: every live segmenter returns a bare list" stale comment ? the diff replaced it. Let me confirm no other file references that stale claim, and check CODE_MAP/docs that describe segmenters always returning bare lists.

39. user (2026-07-06T17:29:00.507Z)

```
backend\pipeline\segmenters\trajectory_hybrid.py:184:        # genuine 0-rally video (which
still returns a bare ``[]``). Every caller
tests\test_results.py:23:    """Legacy segmenters that predate the Result contract still return
a bare list ? the helper
tests\test_worker.py:264:    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match
? status stays "success"
```

40. user (2026-07-06T17:29:02.709Z)

```
trajectory_hybrid OK
worker OK
```

41. assistant (2026-07-06T17:29:14.469Z)

No stale comment remains ? the old "every live segmenter returns a bare list" claim is gone; the remaining references are accurate.

Now let me examine one more important thing: the `# type: ignore[override]` placement. In the diff, the return annotation changed to `"Union[List[Dict[str, Any]], Failure]"` but the `# type: ignore[override]` is on the `def process_video(  # type: ignore[override]` line (line 174). The base ABC declares return `Result[List[Dict[str, Any]]]` = `Union[Success[List], Failure]`. The override returns `Union[List, Failure]` ? `List` is NOT `Success[List]`, so it's still an incompatible override, hence the `# type: ignore[override]` is still needed and correctly kept. Good.

Now, is the `# type: ignore[override]` now actually still triggering (i.e., not an unused-ignore that mypy would flag with `warn_unused_ignores`)? The parameter signature also differs from the base (`player_pool: Optional[List[str]]` vs base `player_pool: List[str] = None`), and the return type differs. So the override ignore is still warranted. Since the task says mypy is clean, this is fine.

Let me check one final potential real issue: the `Union[...]` annotation is a string (forward-ref/quoted). `Failure` is imported at module top (not under TYPE_CHECKING), so the quoted annotation resolves fine at runtime if ever evaluated. And `from __future__ import annotations`? Let me confirm the module doesn't rely on it but the quoting is harmless.

42. user (2026-07-06T17:29:16.104Z)

```
No matches found
```

43. user (2026-07-06T17:29:16.843Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
```

```
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20
21      import concurrent.futures
22      import logging
```

44. assistant (2026-07-06T17:29:44.002Z)

No `from __future__ import annotations`, so the annotation is a quoted string (`"Union[...]"`) that is never evaluated at runtime (function annotations are only evaluated when introspected). The quoting is harmless and mypy resolves it. `Failure` is imported at module level so even runtime introspection would work.

I now have a complete picture. Let me consolidate my findings.

Summary of analysis:

Part A (conventions):
1. **Status token "failed"** ? consistent with ALL consumers: `orchestration.probe_process_cache`/`cached_process_result` check `== "success"`, DB uses "failed", done-marker uses "failed", `_resume_one_remote_job` reads `status` for the failure message. The task's "failed/partial" was loose wording; "failed" is correct and consistent. No "partial" expected anywhere. No violation.
2. **`is_recoverable=True`** ? semantically right (env-not-ready and timeout are retryable). Consistent with gemini (uses default False for hard errors but also plain `Failure(str(e))`), motion (`is_recoverable=False` for "could not open video" ? a hard error). A timeout/healthcheck being recoverable=True is defensible. Note: no consumer currently branches on `is_recoverable`, so it's advisory. No violation.
3. **Widened annotation + kept `# type: ignore[override]`** ? correct. The override is still incompatible with the ABC (`List` vs `Success[List]`), ignore is warranted. Comments accurately describe new behavior; the stale "every live segmenter returns a bare list" comment was removed. No violation.
4. **Twin/base parity** ? CONFIRMED. `process_video` lives only in the base `TrajectoryHybridSegmenter`; wasb/tracknet/fusion subclasses override only `_build_runner`/`_build_native_runner`/enrichment seams, never `process_video`. Fix applies to all three.

Part B (tests): The tests are solid. But I found real GAPS worth reporting.

Let me finalize the findings. The main genuine items:

1. **Test gap (medium):** No end-to-end worker test flows a REAL `TrajectoryHybridSegmenter` Failure (healthcheck or no-trajectory) through `cmd_infer` ? the worker test uses a fake `_FailSeg`, and the segmenter unit tests stop at the segmenter boundary. A regression where `cmd_infer` stops branching on the real segmenter's Failure type wouldn't be caught by either side. This is a legitimate observation but is a coverage-completeness point, not a defect ? the two halves ARE each tested. I'll report as low/medium test gap.

2. **Convention/scope inconsistency (medium, real):** In `trajectory_hybrid.process_video`, the missing-API-key path (line 219, with `skip_ai_handoff=False`) still returns bare `[]`. This is a genuine detector/config FAILURE that "bails BEFORE detection" (the worker's own warning text at line 343 explicitly names it), yet it is now written as a 0-segment `status="success"` ? exactly the wf294 silent-failure class. This is a real residual silent-failure leak. It's pre-existing (not a regression introduced by the diff), and arguably out of the stated 2-path scope, but it's directly in the same failure class the fix targets and the fix's own comments claim "a DETECTOR failure ... returns a Failure so it is NOT confused with a genuine 0-rally video" ? which overstates coverage. Worth reporting as a real gap.

Let me double check the missing-key path is reachable in cloud serving (where wf294 happened). Cloud serving forces `indexing.detector_impl=native` but does NOT force `skip_ai_handoff`. If

`${AI_PROVIDER}_API_KEY` is unset on the L4 container with `skip_ai_handoff=False`, `process_video` returns `[]` at line 219 ? worker writes status="success", 0 segments. That's a real silent-failure identical in shape to wf294. Confirmed reachable and relevant.

Let me verify the `test_cmd_infer_validation_failure_returns_2` gap ? it doesn't assert the new report.json/status. Minor.

Let me now check whether the no-api-key case is actually guarded elsewhere in cloud serving (startup check) so it can't reach process_video with a missing key. The worker warning text says "cloud-serving now fails this at startup".

45. user (2026-07-06T17:29:45.415Z)

```
backend\config\presets.py:40:# ``indexing.{detector_impl,skip_ai_handoff}`` and
``storage.backend``. Later milestones extend
backend\config\presets.py:48:        "indexing": {"skip_ai_handoff": True},
backend\config\presets.py:59:            "skip_ai_handoff": False,
backend\config\presets.py:67:        "indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
backend\config\models.py:267:    skip_ai_handoff: bool = False
backend\config\models.py:441:    (``indexing.detector_impl`` / ``skip_ai_handoff``,
``storage.backend``) in the loader.
backend\config\startup.py:261:        # The AI-handoff key: with skip_ai_handoff=False the
segmenter demands
backend\config\startup.py:262:        # ${AI_PROVIDER}_API_KEY at rally time and bails to an
EMPTY result when it is
backend\config\startup.py:266:        skip_ai = bool(getattr(indexing, "skip_ai_handoff",
False))
backend\config\startup.py:268:        key_var = f"{provider.upper()}_API_KEY"
backend\config\startup.py:275:                    f"(indexing.skip_ai_handoff=False) but
{key_var} is not set ? the segmenter "
backend\config\startup.py:277:                    f"(e.g. a Secret Manager secret) or set
indexing.skip_ai_handoff=true for "
backend\config\loader.py:253:# Deliberately MINIMAL: the `truly-local` PROFILE preset
(presets.py) supplies skip_ai_handoff,
```

46. user (2026-07-06T17:29:48.304Z)

```
250                     )
251                 )
252             elif storage_backend == "gcs" and not _creds_discoverable(e):
253                 checks.append(
254                     StartupCheck(
255                         LEVEL_WARN,
256                         "CREDS_UNCONFIRMED",
257                         "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying
on Application "
258                         "Default Credentials. The GCS backend will fail loudly at init if
ADC can't resolve.",
259                     )
260                 )
261             # The AI-handoff key: with skip_ai_handoff=False the segmenter demands
262             # ${AI_PROVIDER}_API_KEY at rally time and bails to an EMPTY result when it is
263             # missing ? a silent 0-rally "success" AFTER the GPU pass is paid for (observed
on
264             # the first real cloud-serving litmus, 2026-07-03). The env cannot appear later
in
265             # a running container, so this is a zero-false-positive boot-time fatal.
266             skip_ai = bool(getattr(indexing, "skip_ai_handoff", False))
267             provider = str(getattr(config, "ai_provider", "gemini") or "gemini")
268             key_var = f"{provider.upper()}_API_KEY"
269             if not skip_ai and not e.get(key_var):
270                 checks.append(
271                     StartupCheck(
272                         LEVEL_ERROR,
```

```
273                         "AI_HANDOFF_KEY_MISSING",
274                         f"profile 'cloud-serving' runs the {provider} handoff "
275                         f"(indexing.skip_ai_handoff=False) but {key_var} is not set ? the
segmenter "
276                         f"would bail to an empty result after the paid GPU pass. Mount the
key "
277                         f"(e.g. a Secret Manager secret) or set
indexing.skip_ai_handoff=true for "
278                         "the fully-local candidates-as-rallies mode.",
279                     )
280                 )
281             if device == "cpu":
282                 checks.append(
283                     StartupCheck(
284                         LEVEL_WARN,
285                         "CLOUD_CPU_DEVICE",
286                         "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
287                         "detection will not use the GPU.",
288                     )
289                 )
290             elif cuda_available is False and device in ("cuda", "auto"):
291                 checks.append(
292                     StartupCheck(
293                         LEVEL_WARN,
294                         "CLOUD_CUDA_NOT_VISIBLE",
295                         "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
296                     )
297                 )
298
299         elif profile == "cpu-mock":
```

47. assistant (2026-07-06T17:30:07.975Z)

Important context: The missing-API-key silent-failure is guarded at STARTUP for the `cloud-serving` profile (a `LEVEL_ERROR` fatal, `AI_HANDOFF_KEY_MISSING`). So in cloud serving specifically, a missing key aborts the worker at boot (rc 2 via `_run_startup_checks` ? `_fail(2)` ? status="failed" report). That significantly mitigates my concern #2 for the primary wf294 environment. However, this startup guard is only for the `cloud-serving` profile AND only catches the missing-key case ? the OTHER bare-`[]` failure paths (bad sport, unknown provider, invalid video duration, NotImplementedError for a native-unavailable family) are not guarded and would still write status="success". But those are genuinely pre-existing and further out of scope; the invalid-duration/bad-sport cases are edge.

Given the startup guard mitigates the key case for cloud-serving, and these are all pre-existing (not regressions from this diff), I'll report the residual-leak observation at low severity as an honest scoping note rather than a defect the diff introduced ? the diff's comment does overclaim ("A DETECTOR failure ... returns a Failure"). The comment scoping is the reportable part.

Let me do a final check ? verify the diff doesn't break the `test_pipeline_no_api_key_returns_empty` test (line 108 still asserts `== []`), confirming the no-key path intentionally still returns `[]`. It does. So the diff deliberately left that as bare `[]`, and there's a test locking that. That makes the residual-leak a conscious scoping decision, lowering its severity to a documentation/comment-accuracy note.

Now let me finalize. My concrete findings:

1. **[low] Comment over-claims coverage** in `trajectory_hybrid.py:181-186` ? the new docstring says a detector Failure "returns a Failure so it is NOT confused with a genuine 0-rally video," but the missing-API-key path (line 219), bad-sport (194), unknown-provider (226), invalid-duration (231), and native-unavailable (244) paths still return bare `[]` and are written as status="success" by the worker. These are genuine non-detection failures in the same silent-success class. The cloud-serving startup guard only covers the missing-key case for one profile.

Real but pre-existing/out-of-scope; the reportable defect is the comment implying broader coverage than delivered.

2. **[low] Test gap:** no end-to-end worker test drives a REAL `TrajectoryHybridSegmenter` Failure through `cmd_infer` (the worker test uses fake `_FailSeg`; segmenter tests stop at the segmenter boundary). A regression in `cmd_infer`'s `isinstance(result, Failure)` wiring vs. the real segmenter return type wouldn't be caught. Both halves are individually tested, so this is completeness, not a hole.

3. **[low] Test gap:** `test_cmd_infer_validation_failure_returns_2` (test_worker.py:316) now exercises the new `_fail(2)` report-writing path but asserts only `rc == 2`; it does not assert the newly-written report.json has status="failed". The validation-failure branch's new artifact-write is thus unlocked by tests (only the detector-failure rc-3 path asserts the report). A regression writing status="success" (or nothing) on validation failure would pass.

These are all low severity ? the core fix is correct and well-tested. Let me report.

48. user (2026-07-06T17:30:35.285Z)

Structured output provided successfully